Explications du code

TIC315 — Projet 4, jeu d'échecs en ligne. Ce fichier explique simplement chaque partie du code, au fur et à mesure.

Comment le code est rangé

```
code/
├── pom.xml      la liste de ce dont le projet a besoin (Maven)
├── mvnw         lance Maven sans avoir à l'installer
└── metier/      le noyau du jeu : aucune trace de Spring ni du web
    ├── src/main/java/   le code
    └── src/test/java/   les tests JUnit
```

Pour tout compiler et lancer les tests, depuis le dossier `code/` :

```
./mvnw test
```

Partie 1 : les joueurs et le classement

Dossier : `metier/src/main/java/echecs/metier/joueur/`. On commence ici parce que ce package ne dépend d'aucun autre.

Classe	Son rôle
`Joueur`	Un compte : pseudo, e-mail, état, et un classement par famille de cadence
`Classement`	Les points d'un joueur dans une famille. Tout le monde démarre à 1500
`CalculClassement`	Une interface : « donne-moi la variation de points », sans dire comment
`CalculElo`	La formule choisie pour calculer cette variation
`EtatCompte`	`ACTIF` ou `SUSPENDU`
`CategorieCadence`	`BULLET`, `BLITZ` ou `RAPIDE`

Pourquoi ces choix

Pourquoi `CalculClassement` **et** `CalculElo` **sont séparés ?** C'est le patron Stratégie. La partie demandera une variation de points sans connaître la formule. Pour en changer un jour, on ajoutera une classe sans toucher à la partie.

Pourquoi le joueur a un indicateur `occupe` **?** Un joueur ne peut avoir qu'une partie à la fois (RG11). `Joueur` ne connaît pas ses parties, sinon `joueur` et `partie` dépendraient l'un de l'autre. C'est donc la partie qui l'occupe en démarrant et le libère à la fin.

Pourquoi une `EnumMap` **pour les classements ?** Elle associe exactement un `Classement` à chaque `CategorieCadence`. C'est la traduction en Java de l'association qualifiée du diagramme de classes. Les trois classements sont créés avec le joueur, donc jamais absents.

Pourquoi des exceptions ? Une erreur de programmation doit se voir tout de suite plutôt que fausser une partie. Occuper un joueur déjà en partie lève `IllegalStateException` ; créer un joueur sans pseudo lève `IllegalArgumentException`.

La formule Elo, avec un exemple

Un joueur à 1500 bat un joueur à 1480 :

1. Son score attendu est `1 / (1 + 10^((1480 - 1500) / 400))`, soit environ 0,53.
2. Il a marqué 1 (une victoire). Sa variation est `20 × (1 - 0,53)`, soit environ 9,4.
3. On arrondit : il gagne **9 points**, et le perdant en perd **9**.

Battre un joueur beaucoup plus fort rapporte presque 20 points. Battre un joueur beaucoup plus faible ne rapporte presque rien.

Ce que vérifient les tests

Fichier de test	Ce qu'il vérifie
`CalculEloTest`	La formule : victoire, défaite, nulle, joueurs de même niveau, écart important, valeurs refusées
`JoueurTest`	1500 points au départ, classements indépendants (RG6), une partie à la fois (RG11), suspension (RG8), champs obligatoires

Le nom d'un test décrit ce qu'il vérifie, et cite la règle de gestion quand il y en a une : `unCompteSuspenduNePeutPasJouer_RG8`.